

Project: Wealth Management System

1. Overview

The **Wealth Management System (WMS)** is a web-based application aimed at helping financial institutions and individual advisors manage their clients' investment portfolios. The system facilitates onboarding new clients, creating investment plans, tracking portfolio performance, and generating comprehensive reports.

Built using the **MVC architecture**, the system is designed to work seamlessly with both **Java (Spring MVC)** and **.NET (ASP.NET Core MVC)** frameworks, ensuring compatibility and flexibility in diverse programming environments.

2. Core Modules Description

2.1 Client Onboarding

Handles new client registration, verification, and profile creation.

2.2 Investment Planning

Enables financial advisors to design tailored investment strategies for clients.

2.3 Portfolio Management

Monitors investment portfolios, calculates returns, and provides rebalancing recommendations.

2.4 Reporting

Generates detailed reports on investment performance and market trends.

2.5 Compliance Management

Ensures adherence to financial regulations and maintains audit logs.

3. Module-Level Design

3.1 Client Onboarding Module

Purpose: Registers new clients and manages their profiles.

- **Controller:**
 - ClientController
 - registerClient()
 - updateClientProfile()
 - getClientDetails()
- **Service:**
 - ClientService
 - Validates client data and interacts with the database.
- **Model:**

- **Client Entity**
 - Attributes:
 - clientId (PK)
 - name
 - email
 - phone
 - address
 - dateOfBirth

3.2 Investment Planning Module

Purpose: Designs investment strategies based on client goals and risk appetite.

- **Controller:**
 - InvestmentPlanController
 - createPlan()
 - updatePlan()
 - getPlanDetails()
- **Service:**
 - InvestmentPlanService
 - Implements algorithms for plan creation and optimization.
- **Model:**
 - **InvestmentPlan Entity**
 - Attributes:
 - planId (PK)
 - clientId (FK)
 - investmentObjective
 - riskAppetite
 - allocationDetails

3.3 Portfolio Management Module

Purpose: Monitors and rebalances investment portfolios.

- **Controller:**
 - PortfolioController
 - viewPortfolio()
 - rebalancePortfolio()
 - getPortfolioPerformance()
- **Service:**
 - PortfolioService
 - Calculates returns and suggests rebalancing strategies.
- **Model:**
 - **Portfolio Entity**
 - Attributes:
 - portfolioId (PK)
 - clientId (FK)
 - totalValue
 - allocationSummary
 - lastUpdated

3.4 Reporting Module

Purpose: Generates periodic reports and trend analysis.

- **Controller:**
 - ReportController
 - generateReport()
 - downloadReport()
- **Service:**
 - ReportService
 - Prepares reports using data from the portfolio and market analysis.
- **Model:**
 - **Report Entity**
 - Attributes:
 - reportId (PK)
 - portfolioId (FK)
 - reportDate
 - performanceSummary

3.5 Compliance Management Module

Purpose: Ensures regulatory compliance and maintains audit trails.

- **Controller:**
 - ComplianceController
 - logAuditEvent()
 - getAuditLog()
 - verifyCompliance()
- **Service:**
 - ComplianceService
 - Validates transactions and maintains compliance records.
- **Model:**
 - **AuditLog Entity**
 - Attributes:
 - logId (PK)
 - eventTimestamp
 - eventDescription
 - userId
 - complianceStatus

4. Database Schema

4.1 Table Definitions

1. Client Table

```
CREATE TABLE Client (  
  clientId INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100),  
  phone VARCHAR(15),
```

```
address TEXT,  
dateOfBirth DATE  
);
```

2. InvestmentPlan Table

```
CREATE TABLE InvestmentPlan (  
    planId INT AUTO_INCREMENT PRIMARY KEY,  
    clientId INT,  
    investmentObjective VARCHAR(200),  
    riskAppetite ENUM('Low', 'Medium', 'High'),  
    allocationDetails JSON,  
    FOREIGN KEY (clientId) REFERENCES Client(clientId)  
);
```

3. Portfolio Table

```
CREATE TABLE Portfolio (  
    portfolioId INT AUTO_INCREMENT PRIMARY KEY,  
    clientId INT,  
    totalValue DECIMAL(15, 2),  
    allocationSummary JSON,  
    lastUpdated DATE,  
    FOREIGN KEY (clientId) REFERENCES Client(clientId)  
);
```

4. Report Table

```
CREATE TABLE Report (  
    reportId INT AUTO_INCREMENT PRIMARY KEY,  
    portfolioId INT,  
    reportDate DATE,  
    performanceSummary JSON,  
    FOREIGN KEY (portfolioId) REFERENCES Portfolio(portfolioId)  
);
```

5. AuditLog Table

```
CREATE TABLE AuditLog (  
    logId INT AUTO_INCREMENT PRIMARY KEY,  
    eventTimestamp TIMESTAMP,  
    eventDescription VARCHAR(255),  
    userId INT,  
    complianceStatus ENUM('Pass', 'Fail')  
);
```

5. Local Deployment Details

1. Environment Setup:

- Install MySQL or SQL Server for database management.
- Install Java (JDK) or .NET SDK (ASP.NET Core).

- Configure database connections in application.properties (Java) or appsettings.json (.NET).

2. Deployment Steps:

- Clone the repository.
- Build the project using Maven (Java) or Visual Studio (ASP.NET Core).
- Run the application server.
- Access the application at <http://localhost:8080> (Java) or <http://localhost:5000> (.NET).

6. Conclusion

The **Wealth Management System** streamlines the process of investment management, from onboarding clients to ensuring compliance. Its modular design ensures scalability, adaptability, and ease of maintenance, making it an ideal solution for financial institutions and advisors.